

Matrix Dimensions in Deep Neural Networks

CS4152 — Deep Learning and Neural Networks

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

November 22, 2025

Outline

- 1 Network Structure
- 2 Single-Example Dimensions
- 3 Vectorized Dimensions
- 4 Debugging Strategy
- 5 Conclusion

Layer Sizes in a Deep Network

- Example network with 5 layers (excluding input layer).
- Layer dimensions:
 - $n^{[0]} = 2$ (input)
 - $n^{[1]} = 3$
 - $n^{[2]} = 5$
 - $n^{[3]} = 4$
 - $n^{[4]} = 2$
 - $n^{[5]} = 1$ (output)
- Total layers: $L = 5$.

Dimensions for a Single Example

- Forward propagation step:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

- Required shapes:

- $A^{[l-1]}$: $n^{[l-1]} \times 1$
- $W^{[l]}$: $n^{[l]} \times n^{[l-1]}$
- $b^{[l]}$: $n^{[l]} \times 1$
- $Z^{[l]}, A^{[l]}$: $n^{[l]} \times 1$

- Ensures matrix multiplication consistency.

Example Shapes

- $W^{[1]}$: 3×2
- $W^{[2]}$: 5×3
- $W^{[3]}$: 4×5
- $W^{[4]}$: 2×4
- $W^{[5]}$: 1×2
- Bias vectors follow:

$$b^{[l]} \in \mathbb{R}^{n^{[l]} \times 1}$$

Vectorized Forward Propagation

- For m training examples:

$$A^{[0]} = X \in \mathbb{R}^{n^{[0]} \times m}$$

- New shapes:

$$Z^{[l]}, A^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$$

- Matrix multiplication:

$$W^{[l]}(n^{[l]} \times n^{[l-1]}) \cdot A^{[l-1]}(n^{[l-1]} \times m) = Z^{[l]}(n^{[l]} \times m)$$

- Bias broadcasting automatically expands $b^{[l]}$.

Broadcasting of Bias Terms

- $b^{[l]}$: $n^{[l]} \times 1$
- Broadcast to:

$$n^{[l]} \times m$$

- Added element-wise to $Z^{[l]}$.

Dimension-Based Debugging

- Check dimensions of every variable before coding.
- Ensure:
 - $\dim(W^{[l]}) = \dim(dW^{[l]})$
 - $\dim(b^{[l]}) = \dim(db^{[l]})$
 - $\dim(Z^{[l]}) = \dim(A^{[l]})$
- Prevents common bugs in both forward and backward propagation.

Key Takeaways

- Matrix dimensions dictate the architecture structure.
- Vectorization changes only $Z^{[l]}$, $A^{[l]}$, and X dimensions.
- Dimension discipline is essential for bug-free implementation.
- Forms the foundation for backpropagation and computational graphs.